FRED TALKS

19th May 2026

CONTENTS

How big are our embeddings now and why?

★❤☆ VICKI BOYKIS ★❤☆ · 10 SEP 2025 · SOURCE

A few years ago, I wrote a paper on embeddings. At the time, I wrote that 200-300 dimension embeddings were fairly common in industry, and that adding more dimensions during training would create diminishing returns for the effectiveness of your downstream tasks (classification, recommendation, semantic search, topic modeling, etc.)

I wrote the paper to be resilient to changes in the industry since it focuses on fundamentals and historical context rather than libraries or bleeding edge architectures, but this assumption about embedding size is now out of date and worth revisiting in the context of growing embedding dimensionality and embedding access patterns.

As a quick review, embeddings are compressed numerical representations of a variety of features (text, images, audio) that we can use for machine learning tasks like search, recommendations, RAG, and classification. The size of the embedding is how many features our item has.

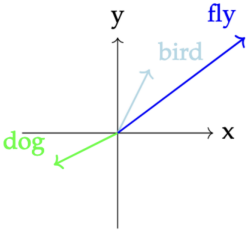
For example, let's say we have two butterflies. We can compare them among many dimensions, including wingspan, wing color, number of antennae. Let's say we have a 3-dimensional feature for a butterfly, it might look like this.

	wing_color	wing_span	antennae
butterfly_1	blue	10 cm	1
butterfly_2	red	20cm	2
butterfly_3	blue	15cm	1

Doing a visual vibe scan, we can eyeball the data and see that `butterfly_1` and `butterfly_3` are more similar to each other than to `butterfly_2` because their features are closer together.

But butterflies are 3-dimensional animals and some of these features are numerical. When we talk about embeddings in industry these days, we generally mean trying to understand the properties of text, so how does this concept work with words? We can't directly compare words like "bird" and "butterfly" or "fly" in a given text but we can compare their numerical representations if we map them into the same shared space. We can see that "bird" and "flying" are more similar to each other than to "dog" through their numerical representations.

Intuitively, we know this is true, but how do we artificially create these relationships?



There are different ways of creating text embeddings from a given word, all of which rely on analyzing that word in relation to the words around it in a given corpus of data. We can use traditional count-based approaches like TF-IDF based on term frequency in documents, or statistical approaches like PCA or LSA.

With the advent of deep learning models, we started learning representations generated from models like Word2Vec that maximized the probability that a left-out word would be next to other given words in the training dataset.

When we learn the embedding representation of a given word using probabilistic models, we are comparing how similar these words are to other words. Each feature is not an explicit feature like "wing color", but rather a vibe-based latent representation in the latent space that doesn't have a clear explanation.

For example, one dimension might be "this word is an action word" or maybe "this word is related to other words about food", but we generally don't know exactly what the model thinks each feature represents. In fact, this is a fascinating area of study we are just starting to understand how these latent representations work through ideas like control vectors, a concept that Anthropic explored in the famous Golden Gate Claude paper.

When we train a model, embedding size is initialized as a hyperparameter before model training, and we iterate on the size depending on our downstream evaluations after training. Picking the right hyperparameter is (alchemy) a combination of art and science and depends on optimizing training throughput, final embedding storage size, and the performance of the embedding on your downstream task both qualitatively and wrt to latency of performing search on embeddings of different sizes.

Since previous generations of models were smaller and trained in-house, hyperparameters were usually not published by companies, and as such, there was no standard agreement on embedding size. We generally, as an industry, understood that somewhere around 300 dimensions for a given embedding model might be enough to compress all the nuance of a given textual dataset. 300 was the number of dimensions typically used by earlier models like Word2Vec and GloVE.

After the publication of the attention paper BERT was released in 2018. This model's architecture introduced embeddings of 768 dimensions. Although previous RNN and LSTM models had been trained on GPUs, BERT was one of the first larger embedding models to be trained on GPUs (and TPUs), which meant that GPU optimization now became increasingly important.

The key behind training Transformer models efficiently is the ability to efficiently move data onto GPU and parallelize their matrix multiplication operations between several pipelines, aka attention heads, where each attention head can focus on understanding and defining a different part of the embedding feature space. As such, the embedding needs to be able to be partitioned evenly between the number of attention heads.

BERT has 12 attention heads, so 768 dimensions was selected from a combination of trial and error and efficiently parallelizing computation to "attend" to different parts of the feature space, meaning that, in model training, each head operates on a 64-dimensional subspace of the original 768-dimensional input embedding. This itself comes from the Transformer paper, where each sub-embedding size per head is commonly chosen as 64.

As a result, many BERT-style models, and related model families, used 768 as a baseline for embedding dimensions. Although it had a much larger training dataset, GPT-2 also implemented 768. And, although CLIP uses an embedding size of **768** as derived from the Vision Transformer architecture that CLIP uses for its image encoder, and for consistency, the text encoder also uses this dimension size[^1].

Even though training cycles for BERT were fairly small (4 days for the original BERT model) compared to the months-long pretraining processes that LLMs require these days, it was still hard computationally to infer these embedding sizes, even with GPU optimizations. For BERT, for example,

```
Finding the most similar pair in a collection of 10,000 sentences req
```

So in 2019, UKP created and optimized a model, SBERT, focusing on sentence-level representations, which unlocked faster inference, and Minilm became the standard baseline model for document-level chunk embeddings at 384 dimensions and still performs extremely well for a number of tasks.

What else changed was the rise of the open-source HuggingFace platform where model artifacts could be shared. Previously, they were either hosted in undiscoverable repos in GitHub or locked away in scientific repositories missing metadata. The rise of HuggingFace as a platform and the `transformers` library as a centralized API into training and inference for PyTorch-based models unlocked a level of standardization: now, many more people could simply download and replicate the existing models instead of rebuilding arcane research code from scratch. This led to much more standard embedding and architecture sizes used both in academia and industry.

Although 768 held for a fairly long time, with upwards pressure from market competition on model sizes as the result of the release of GPT-2 (the backbone of ChatGPT), we started seeing standard embedding sizes increase.

Part of this was the fact that companies no longer had to infer their own embeddings. Previously, embeddings had been custom-learned in labs or in companies whose core competency was processing large amounts of information that could be culled in retrieval through search or recommendation.

But now, with the advent of ChatGPT and API-based model availability, embeddings became a commodity available with a GET request, and the most popular embeddings became OpenAI's, which used 1536 dimensions, in line with GPT-3, which used much more training data than any previous model. (570GB versus GPT-2 which was 40GB, and 96 attention heads.)

It used to be the case that people only learned or fine-tune their own embeddings, but now all of the major AI providers have their own hosted sets of embeddings. Particularly common ones are OpenAI, with Cohere and Nomic close behind. Google also recently released embeddings for Gemini, with both API and local versions being available.

In addition to standardization via HuggingFace and APIs, MTEB, which benchmarks embeddings publicly, has grown as a resource where you can compare embedding models.

If we look at MTEB, the embedding benchmark today, we can see embedding sizes anywhere from our classic 768 to 4096 and beyond (note all of them are neatly divisible by 2, in line with architectural constraints)

Rank (Bor…	Model	Zero-shot	Embedding D…
1	gemini-embedding-001	99%	3072
2	Qwen3-Embedding-8B	99%	4096
3	Qwen3-Embedding-4B	99%	2560
4	Qwen3-Embedding-0.6B	99%	1024
5	Linq-Embed-Mistral	99%	4096
6	gte-Qwen2-7B-instruct	⚠ NA	3584
7	multilingual-e5-large-instruct	99%	1024
8	SFR-Embedding-Mistral	96%	4096
9	text-multilingual-embedding-002	99%	768
10	GritLM-7B	99%	4096
11	GritLM-8x7B	99%	4096

Finally, another consideration has been the change in the landscape with vector databases, which used to be huge, are now becoming more commoditized features in software and platforms like Postgres, S3, and Elasticsearch, leading to less out of the box necessity in vector storage and performance tuning.

With the constant upward pressure on embedding sizes not limited by having to train models in-house, it's not clear where we'll slow down: Qwen-3, along with many others is already at 4096. It does appear that we are beginning to trim down on growth. OpenAI implemented a concept called matryoshka representation learning that trains embeddings with the "most important" concepts first, and additional embeddings adding incremental gains, meaning that an embedding learned in 1024 dimensions might be just as useful in 64, as long as the first 64 dimensions compress most of the information efficiently, and also making sure we re-normalize them. There are also research that indicates that not all embeddings are necessary in retrieval + search tasks, and that we can in fact, in some cases, truncate 50% of them anyway.

It's been fascinating to watch the rise of dimensionality and the constant struggle in tradeoffs between creating ever-larger models and then the need for those embedding sizes to perform at inference and storage time, aka continuously coming up against the age-old machine learning tradeoff of recall versus precision, and the engineering tradeoff of hardware limitations versus software versus business considerations.

As these architectures have matured and internal models have become inference points of public-facing paid APIs, embeddings have gone from a mysterious byproduct of internal machine learning systems in companies with lots of data to a commodity used across many AI-powered applications across numerous application stacks.

All in all, it's been so much fun watching this space evolve, even if it means it looks like I'll be having to update my paper over and over again.

Six Years Perfecting Maps on watchOS

DAVID SMITH · 17 MAY 2026 · SOURCE

I love going on wilderness adventures. I am rarely happier than when I am far off into the mountains without a soul in sight. As a result, I have spent a lot of time learning how to safely explore and navigate when I'm away from civilization. The most important habit I've found for not getting lost is to be *very* regular in checking your location as you go, and the best way I've found to do that is to have a map on my wrist.

For more than **six** years I've been working towards creating the best possible mapping experience on the Apple Watch. With yesterday's launch of Pedometer++ 8, I feel like this design journey has reached a meaningful destination. I would contend that Pedometer++'s watchOS mapping support is the absolute best available on the App Store.

So I wanted to walk through the journey it took to get here.

Early Efforts

I have wanted a good map on my wrist since the Apple Watch launched. This wasn't realistically possible until watchOS 6, which brought SwiftUI to the platform and, for the first time, made *"real"* apps possible. But in those early days, the screens were tiny, and the processors slow. I couldn't *quite* get to where I wanted.

This was my very first attempt that shipped in Pedometer++. These maps were generated completely on the server, which involved sending the relevant workout data roundtrip every time I wanted to refresh the display. This system let me validate the idea, but it was never going to be practically useful for navigation or regular use, and could never work offline.

Custom Mapping Engine

I knew that if I wanted to make progress towards this goal, I'd need to work at a lower level, so I got to work building a fully SwiftUI-native map rendering engine. SwiftUI was the only choice because it's all that watchOS supported, and proved to be helpful for putting maps into widgets, which also only support SwiftUI.

In 2021, I got this engine to a place where I could reliably and performantly render a map on watchOS. With it, I can render any tile-based maps and overlay location information on top.

Map Designs

Next came the question of how best to surface data to users. App design on watchOS is a really fun — but frustrating — challenge. You are designing for a relatively tiny screen, which must be operated one-handed. In this case, I want the user to be able to read the map and use it to navigate, while also having access to other workout-related information.

This began a long series of design attempts, most of which (if I'm being honest) were kinda awful.

Bad designs

In the end, I settled on a "modal" approach where the user can switch between a map screen and a metrics screen using a button on the top-left corner.

Modal Maps

This interface provides one context where the user can freely pan/zoom around the map and another where I can use the more standard watchOS tabbed page interface for metrics and controls. I shipped this to Pedometer++, but there was always something that didn't quite sit right with me about it.

This design felt like a compromise, and not in a good way. I felt that in order to achieve the goal of making the map interactive, I couldn't have the map be part of any UI structure that involved swipes. As the screens on Apple Watches got larger, it felt less needed in order to give the map enough space to be useful.

So I set about trying alternative designs. **SO** many designs.

Many more designs

For a while, I thought that I needed to find a way to put the metrics at the bottom of the screen. However, that would lead to other problems on longer outings or for workouts that aren't navigation-focused. So I kept iterating and came up with even *more* designs.

Even more designs

All of these designs suffered from the same fundamental issue: they required the app to display only a fixed set of fields at a time.

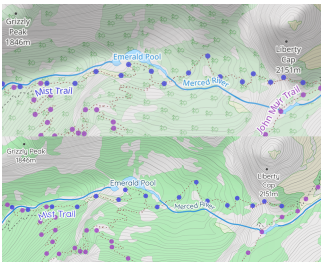
I could make the interface configurable, but one of the *fundamental* rules of watchOS design is that you should avoid any interaction that takes more than a few seconds on the watch. Any user-configurable setup is inherently fiddly, so I didn't like this approach.

Dark Mode, Liquid Glass, & Cartography

Around the same time I was still wrestling with the design challenges of how best to structure the app, Apple announced watchOS 26, and the arrival of Liquid Glass. One of the core design aspects of Liquid Glass is layering stacking elements on top of each other, but another is the types of colors that work best with each other.

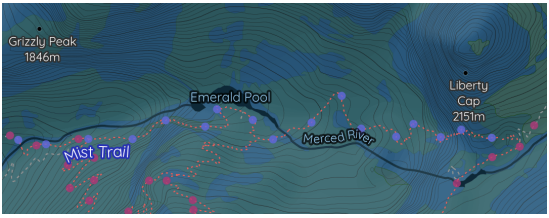
I was previously using Thunderforest Outdoors as my basemap for the app. I love the content this map includes, but when I started overlaying glassy elements over it I found that it wasn't well-suited for Liquid Glass.

So… I commissioned a custom map. Working with the incredible cartographer Andy Allen, we created a completely new basemap that would look fantastic with Liquid Glass.[1]



We simplified the map visually, increased the contrast of the elements, and made the map elements more saturated to prevent them from becoming a muddy mess when shown below glass.

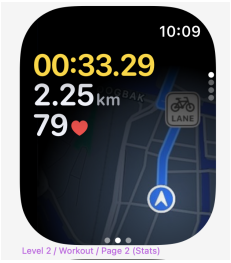
With this work done, I had another opportunity: I could finally have a dark mode variant of the map tiles. While helpful on iOS, this really shines on watchOS. Andy and I really worked toward something which would be incredibly legible at arm's length.



The result of these efforts is that now I have a great map for watchOS… but a design that didn't match that greatness.

Striving for Great

I kept trying. To get me out of my design rut, I enlisted the help of the fantastic designer Rafa Conde. I needed a fresh set of eyes on this and very quickly, this partnership paid off. They proposed a variety of alternative layouts, but when I saw this one I *knew* it was **the one**.



The layering of the metrics on the top-left corner, with the map being the top page of a vertical stack, was the correct answer. This design handles interactivity by requiring a tap on the map first to enter "browse mode".

Tweaking and Polishing

Now that I had the overall concept locked in, the real fun began, actually building the app and dialing in all the details. I fairly quickly took Rafa's concept and turned it into a working prototype. This let me validate the idea in the field… literally. After walking a few hundred miles with it, I was confident it was the correct approach.

Next, I needed to dial in the font and make more subtle design choices.

Map fonts

After a bit more iteration, I arrived at the design that shipped yesterday. It is legible, useful, and (in my humble opinion) beautiful.

It feels really good to be able to cap off this six-year journey with a design I couldn't be more proud of. This screen represents so much accumulated effort and learning. It finally gives me a design which feels native on the platform, but also novel and unique.

Here is the evolution of this design over the last six years:

Postscript: Considering MapKit

While my work on watchOS mapping massively predates the arrival of Apple's MapKit onto the platform, it is probably worth explaining why I decided to do all of this custom work to avoid using it.

Fundamentally, I find that MapKit is great for *basic* uses, but doesn't provide nearly the level of configurability and utility which I want Pedometer++ to offer. For example:

- MapKit on watchOS always shows in dark mode, which generally is a good default, but closes the door on some accessibility and user choice reasons. I needed it to be a user-selectable option.

- While MapKit on watchOS has gotten better over time in terms of what you can do with it, I still find it a bit limiting in terms of animations and overlays.

- MapKit's coverage is improving with regards to topographic contours and trail marking, but there are far too many places where the MapKit map is essentially blank, but I know there should be more rich details available. For example, here is my map vs MapKit at the trailhead of one of my favorite hikes in Scotland.

MapKit comparision

1. I still find it so cool that my work on this allows me to say that I "commissioned a cartographer" to work on something for me. 😁 ↩

Big tech engineers need big egos

SEANGOEDECKE.COM RSS FEED · 14 MAR 2026 · SOURCE

It's a common position among software engineers that big egos have no place in tech[1]. This is understandable - we've all worked with some insufferably overconfident engineers who needed their egos checked - but I don't think it's correct. In fact, I don't know if it's possible to survive as a software engineer in a large tech company without some kind of big ego.

However, it's more complicated than "big egos make good engineers". The most effective engineers I've worked with are simultaneously high-ego in some situations and surprisingly low-ego in others. What's going on there?

Engineers need ego to work in large codebases

Software engineering is shockingly humbling, even for experienced engineers. There's a reason this joke is so popular:

The minute-to-minute experience of working as a software engineer is dominated by *not knowing things* and *getting things wrong*. Every time you sit down and write a piece of code, it will have several things wrong with it: some silly things, like missing semicolons, and often some major things, like bugs in the core logic. We spend most of our time fixing our own stupid mistakes.

On top of that, even when we've been working on a system for years, we still don't know that much about it. I wrote about this at length in *Nobody knows how large software products work*, but the reason is that big codebases are just that complicated. You simply can't confidently answer questions about them without going and doing some research, even if you're the one who wrote the code.

When you have to build something new or fix a tricky problem, it can often feel straight-up impossible to begin, because good software engineers know just how ignorant they are and just how complex the system is. You just have to throw yourself into the blank sea of millions of lines of code and start wildly casting around to try and get your bearings.

Software engineers need the kind of ego that can stand up to this environment. In particular, they need to have a firm belief that they *can* figure it out, no matter how opaque the problem seems; that if they just keep trying, they can break through to the pleasant (though always temporary) state of affairs where they understand the system and can see at a glance how bugs can be fixed and new features added[2].

Engineers need ego to work in big tech companies

What about the non-technical aspects of the job? Nobody likes working with a big ego, right? Wrong. Every great software engineer I've worked with in big tech companies has had a big ego - though as I'll say below, in some ways these engineers were surprisingly low-ego.

You need a big ego to take positions. Engineers love being non-committal about technical questions, because they're so hard to answer and there's often a plausible case for either side. However, as I keep saying, engineers have a duty to take clear positions on unclear technical topics, because the alternative is a non-technical decision maker (who knows even less) just taking their best guess. It's scary to make an educated guess! You know exactly all the reasons you might be wrong. But you have to do it anyway, and ego helps *a lot* with that.

You need a big ego to be willing to make enemies. Getting things done in a large organization means making some people angry. Of course, if you're making lots of people angry, you're probably screwing up: being too confrontational or making obviously bad decisions. But if you're making a large change and one or two people are angry, that's just life. In big tech companies, any big technical decision will affect a few hundred engineers, and one of them is bound to be unhappy about it. You can't be so conflict-averse that you let that stop you from doing it, if you believe it's the right decision. In other words, you have to have the confidence to believe that you're right and they're wrong, even though technical decisions always involve unclear tradeoffs and it's impossible to get absolute certainty.

You need a big ego to correct incorrect or unclear claims. When I was still in the philosophy world, the Australian logician Graham Priest had a reputation for putting his hand up and stopping presentations when he didn't understand something that was said, and only allowing the seminar to continue when he felt like he understood. From his perspective, this wasn't rude: after all, if *he* couldn't understand it, the rest of the audience probably couldn't either, and so he was doing them a favor by forcing a more clear explanation from the speaker.

This is obviously a sign of a big ego. It's also a trait that you need in a large tech company. People often nod and smile their way past incorrect technical claims, even when they suspect they might be wrong - assuming that they've just misunderstood and that somebody else will correct it, if it's truly wrong. **If you are the most senior engineer in the room, correcting these claims is your job.**

If everyone in the room is so pro-social and low-ego that they go along to get along, decisions will get made based on flatly incorrect technical assumptions, projects will get funded that are impossible to complete, and engineers will burn weeks or months of their careers vainly trying to make these projects work. You have to have a big enough ego to think "actually, I think I'm right and everyone in this room is confused", even when the room is full of directors and VPs.

Sometimes you need to put your ego aside

All of this selects for some pretty high-ego engineers. But in order to actually *succeed* in these roles in large tech companies, you need to have a surprisingly low ego at times. **I think this is why *really* effective big tech engineers are so rare: because it requires such a delicate balance between confidence and diffidence.**

To be an effective engineer, you need to have a towering confidence in your own ability to solve problems and make decisions, even when people disagree. But you also need to be willing to instantly subordinate your ego to the organization, when it asks you to. At the end of the day, your job - the reason the company pays you - is to execute on your boss's and your boss's boss's plans, whether you agree with them or not.

Competent software engineers are allowed quite a lot of leeway about *how* to implement those plans. However, they're allowed almost no leeway at all about the plans themselves. In my experience, being confused about this is a common cause of burnout[3]. Many software engineers are used to making bold decisions on technical topics and being rewarded for it. Those software engineers then make a bold decision that disagrees with the VP of their organization, get immediately and brutally punished for it, and are confused and hurt.

In fact, **sometimes you just get punished and there's nothing you can do.** This is an unfortunate fact of how large organizations function: even if you do great technical work and build something really useful, you can fall afoul of a political battle fought three levels above your head, and come away with a *worse* reputation for it. Nothing to be done! This can be a hard pill to swallow for the high-ego engineers that tend to lead really useful technical projects.

You also have to be okay with having your projects cancelled at the last minute. It's a very common experience in large tech companies that you're asked to deliver something quickly, you buckle down and get it done, and then right before shipping you're told "actually, let's cancel that, we decided not to do it". This is partly because the decision-making process can be pretty fluid, and partly because many of these asks originate from off-hand comments: the CTO implies that something might be nice in a meeting, the VPs and directors hustle to get it done quickly, and then in the next meeting it becomes clear that the CTO doesn't actually care, so the project is unceremoniously cancelled[4].

Final thoughts

Nobody likes to work with a bully, or with someone who refuses to admit when they're wrong, or with somebody incapable of empathy. But you really do need a strong ego to be an effective software engineer, because software engineering requires you to spend most of your day in a position of uncertainty or confusion. If your ego isn't strong enough to stand up to that - if you don't believe you're good enough to power through - you simply can't do the job.

This is particularly true when it comes to working in a large software company. Many of the tasks you're required to do (particularly if you're a senior or staff engineer) require a healthy ego. However, there's a kind of catch-22 here. If it insults your pride to work on silly projects, or to occasionally "catch a stray bullet" in the organization's political fights, or to have to shelve a project that you worked hard on and is ready to ship, you're too high-ego to be an effective software engineer. But if you can't take firm positions, or if you're too afraid to make enemies, or you're unwilling to speak up and correct people, you're too low-ego.

Engineers who are low-ego in general can't get stuff done, while engineers who are high-ego in general get slapped down by the executives who wield real organizational power. The most successful kind of software engineer is therefore a chameleon: low-ego when dealing with executives, but high-ego when dealing with the rest of the organization[5].

1. What do I mean by "ego", in this context? More or less the colloquial sense of the term: a somewhat irrational self-confidence, a tendency to believe that you're very important, the sense that you're the "main character", that sort of thing

   ↩

2. Why is this "ego", and not just normal confidence? Well, because of just how murky and baffling software problems feel when you start working on them. You really do need a degree of confidence in yourself that feels unreasonable from the inside. It should be

obvious, but I want to explicitly note that you don't *just* need ego: you also have to be technically strong enough to actually succeed when your ego powers you through the initial period of self-doubt.

↩

3. I share the increasingly-common view that burnout is not caused by working too hard, but by hard work unrewarded. That explains why nothing burns you out as hard as being punished for hard work that you expected a reward for.

↩

4. It's more or less exactly this scene from Silicon Valley.

↩

5. This description sounds a bit sociopathic to me. But, on reflection, it's fairly unsurprising that competent sociopaths do well in large organizations. Whether that kind of behavior is worth emulating or worth avoiding is up to you, I suppose.

↩

A new EDIT tool for LLM agents

<ANTIREZ> · 19 MAY 2026 · SOURCE

```
Right now I'm working to an agent for my DS4 project. Local inference

This means, basically, that just using line numbers is very fragile:

The READ and SEARCH tools return something like that:

  10:Q8fA int count = 10;
  11:rA3_ if (count > limit) {
  12:Kq9z     count = limit;
  13:PX0b }
```

So there are line numbers and tags. The tag is 4 chars, on average 2.

```
  {
    "tool": "edit",
    "path": "/tmp/example.c",
    "line": 10,
    "tag": "Q8fA",
    "new": "int count = 11;"
  }
```

Or, multi line, like this:

```
  {
    "tool": "edit",
    "path": "/tmp/example.c",
    "lines": "11:rA3_\n12:Kq9z\n13:PX0b",
    "new": "if (count > limit)\n    return limit;"
  }
```

The saving is significant especially when the agent is deleting big a

The interesting thing is that DeepSeek v4 Flash is able to use this t

: